

Laborator 4- Algoritmi

Pentru un arbore roșu-negru, implementați operația de inserare. Utilizați fișiere header.

Modul de implementare a unui arbore roșu și negru în C se face astfel:

```
typedef struct rb_node {
    enum { red, black } color;
    int key;
    struct rb_node *left,
                  *right,
                  *parent;
} t_rb_node;

typedef struct {
    t_rb_node *root;
} t_rb_arbore;
```

Pseudocod:

<pre>MAKE_ROOT(RB_ ARBORE A, KEY) Allocate memory for A.root A.root.key := KEY A.root.left := NULL A.root.right := NULL A.root.parent := NULL A.root.color := BLACK RETURN A.root END MAKE_ROOT</pre>	<pre>CREATE_NODE(KEY) Allocate memory for node node.key := KEY node.left := NULL node.right := NULL node.parent := NULL node.color := BLACK RETURN node END CREATE_NODE</pre>
<pre>INORDER_WALK(RB_NOD ROOT) IF ROOT <> NULL INORDER_WALK(ROOT.left) PRINT ROOT.key IF ROOT.color == BLACK PRINT "[b]" ELSE PRINT "[r]" END IF INORDER_WALK(ROOT.right) END IF END INORDER_WALK</pre>	<pre>TREE_SEARCH(RB_NOD N, KEY) IF N==NULL OR KEY==N.key THEN RETURN N IF KEY < N.key THEN RETURN TREE_SEARCH(N.left, KEY) ELSE RETURN TREE_SEARCH(N.right, KEY) END IF END TREE_SEARCH</pre>

<pre> RB_INSERT(RB_ARBORE A, RB_NOD Z) RB_NOD y := NULL RB_NOD x := A.root WHILE x <> NULL DO y := x IF Z.key < x.key THEN x := x.left ELSE x := x.right END IF END WHILE Z.parent := y IF y == NULL THEN A.root := Z ELSE IF Z.key < y.key y.left := Z ELSE y.right := Z END IF Z.left := NULL Z.right := NULL Z.color := RED RB_INSERT_FIXUP(A, Z) END RB_INSERT </pre>	<pre> LEFT_ROTATE (RB_ARBORE A, RB_NOD X) RB_NOD y y := X.right X.right := y.left IF y.left <> NULL THEN y.left.parent := X END IF y.parent := X.parent IF X.parent == NULL THEN A.root := y ELSE IF X == X.parent.left THEN X.parent.left := y ELSE X.parent.right := y END IF y.left := X X.parent := y END LEFT_ROTATE </pre>
<pre> RB_INSERT_FIXUP(RB_ARBORE A, RB_NOD Z) WHILE Z.parent <> NULL AND Z.parent.color == RED DO IF Z.parent == Z.parent.parent.left THEN y := Z.parent.parent.right IF y <> NULL AND y.color == RED THEN Z.parent.color := BLACK y.color := BLACK Z.parent.parent.color := RED Z := Z.parent.parent ELSE IF Z == Z.parent.right THEN Z := Z.parent LEFT_ROTATE(A, Z) END IF Z.parent.color := BLACK Z.parent.parent.color := RED RIGHT_ROTATE(A, Z.parent.parent) END IF ELSE (similar cu ramura THEN, doar ca se schimba intre ele right cu left) END IF END WHILE A.root.color := BLACK END RB_INSERT_FIXUP </pre>	

```
MAIN()
  RB_ARBORE T
  RB_NOD r, n
  PRINT "ROOT="
  READ x
  r := MAKE_ROOT(T, x)
  PRINT "Introduceti x pana la citirea 0 \n"
  PRINT "x= "
  READ x
  WHILE x <> 0 DO
    n := CREATE_NODE(x)
    RB_INSERT(T, n)
    PRINT "x= "
    READ x
  END WHILE
  PRINT "In-order walk:"
  INORDER_WALK(r)
  PRINT NEWLINE

  PRINT "Cauta nod cu cheia x= "
  READ x
  n := TREE_SEARCH(r, x)
  IF n <> NULL THEN
    PRINT "Nodul cu cheia "+n.key+ "gasit"
  ELSE
    PRINT "Nodul nu a fost gasit"
  END IF
END MAIN
```